

Tres asistentes, tres personas

Construí tres asistentes que viven en el chat. Comparten el mismo motor, pero uno es mi asistente personal con más de sesenta herramientas, otro atiende al público de uno en uno y el tercero coordina varias voces dentro de un grupo. Aquí van los tres lado a lado, qué hace cada uno en detalle, y por qué son la **base sobre la que sigo construyendo agentes para mi equipo**. Todos los ejemplos están anonimizados.

Si atiendes pacientes o clientes, tarde o temprano te preguntas si un asistente conversacional te sirve. La respuesta honesta es: **depende de la forma**. No es lo mismo un asistente para ti solo que uno que da la cara a desconocidos, ni uno de tú a tú que uno metido en un grupo de veinte personas. Escribí este documento en dos capas: primero, qué hace cada uno —legible para quien decide—; después, cómo están hechos por dentro, para tu asesor técnico.

Los tres, lado a lado

	● Personal	● Soporte 1:1	● Grupos
Para quién	Solo para mí	Público, de uno en uno	Equipos de trabajo

	● Personal	● Soporte 1:1	● Grupos
Canal	Telegram, interfaz web y voz	WhatsApp y Telegram	WhatsApp (grupos)
Identidad	Un SOUL con carácter	SOUL público con límites duros	Un alma por grupo: voz, modelo y KB propios
Memoria	Longitudinal: me conoce con el tiempo	KB de producto + expediente por contacto	Contexto e historia de cada grupo
Cómo participa	Siempre, a mi orden	Responde a quien escribe	Tres modos (callado, proactivo, a orden)
Se administra	Panel web	Panel web	Panel web
Me avisa por	—	Telegram (privado)	Telegram (privado)
Superpoder central	Más de 60 herramientas	Levantar tickets y aprender	Coordinar sin saturar

La tecnología de abajo es casi la misma en los tres; lo que cambia es la forma. Y los tres se configuran desde un **panel web** —alma, bases de conocimiento, modelo, modos de participación y autorizaciones— sin tocar código.

1 • El asistente personal — más de 60 superpoderes

Es solo para mí, con nombre y carácter, y lo alcanzo por Telegram, por una interfaz web y por voz. No es un chatbot que responde preguntas: es un **asistente que actúa**. Hoy suma **más de sesenta herramientas** —lo que llamo sus superpoderes— agrupadas en unas pocas familias:

Comunicación

Resume mi bandeja de correo, me redacta respuestas y traduce sobre la marcha.

Productividad

Recordatorios, tareas y seguimientos; me avisa a tiempo y no deja que algo se caiga.

Conocimiento

Busca en la web, lee documentos e imágenes y responde citando de dónde salió el dato.

Memoria longitudinal

Aprende cómo trabajo y retoma hilos de hace semanas sin que se los repita.

Meta / auto-observación

Revisa mis propias sesiones de trabajo con IA y me devuelve un resumen de avances y pendientes.

Multimodal

Habla y escucha por voz, y entiende fotos y capturas, no solo texto.

La pieza que más me sorprendió es la de **auto-observación**: revisa mis propias sesiones de trabajo con IA y me devuelve, en lenguaje llano, qué avancé y qué quedó pendiente. Un asistente que no solo hace tareas, sino que me ayuda a ver cómo trabajo. Su **memoria es longitudinal**: me conoce con el tiempo y retoma hilos de hace semanas sin que se los repita.

2 • El de soporte 1:1 — atiende al público

Da la cara a desconocidos por WhatsApp y Telegram, de uno en uno. Es el primer contacto: resuelve dudas de producto o servicio y, sobre todo, **no deja caer nada**. Lo que hace, en concreto:

- **Atiende por WhatsApp y Telegram**, los canales donde la gente ya está —sin app nueva que instalar—, y entiende texto, imágenes y documentos.
- **Levanta tickets**: registra cada solicitud, la clasifica y le da folio, para que ninguna consulta se pierda en el hilo.
- **Aprende**: cuando le enseño algo nuevo, lo guarda como un hecho más; si es un dato delicado, queda pendiente de mi revisión antes de usarlo.
- **Consulta la base de conocimiento** (precios, políticas, datos de cursos) antes de responder, en vez de contestar de memoria.

- **Me avisa por Telegram** en privado cuando algo se sale de guion o necesita a un humano: precios en firme, una escalación dudosa, una alerta.

3 • El de grupos — varias voces en un chat

Vive dentro de grupos de trabajo, y es el más delicado porque un chat compartido perdona menos. Lo que lo hace convivible:

- **Un alma, un modelo y una identidad por grupo.** Cada grupo puede tener su propia personalidad, su propio modelo de lenguaje y sus propias bases de conocimiento, versionadas y con prioridad. La misma infraestructura suena distinta en dirección que en soporte.
- **Promueve la participación:** lanza temas, hace preguntas y reanima un hilo que se apaga, en vez de esperar callado a que alguien lo invoque.
- **Mantiene el contexto:** guarda los acuerdos y la historia del grupo, para que la conversación tenga continuidad y no arranque de cero cada vez.
- **Tres maneras de participar:** callado (solo si lo mencionan o citan), proactivo (un clasificador decide si aporta, con umbral y enfriamiento para no ser pesado) o solo-a-orden (nada más el dueño, con un comando).
- **Candado de autorización:** si alguien lo mete a un grupo nuevo, no participa; avisa una vez, me manda un aviso privado y espera a que yo lo autorice.

El alma: la personalidad es un archivo

A cada asistente lo define un **SOUL**: un archivo de texto plano con quién es, cómo habla y qué nunca hace. No es código; es prosa que puedo editar en un minuto, y el asistente la relee sola cada tanto sin reiniciar nada. Ahí vive el detalle que más cambia la percepción: el **registro de habla**.

Suena menor y no lo es. La diferencia entre "¡Hola! Con mucho gusto, ¿en qué puedo ayudarte?" y un simple "Hola, ¿qué necesitas?" es la diferencia entre un

bot y alguien. Mi asistente personal tiene prohibidas frases que huelen a robot ("con gusto", "excelente pregunta", "¿algo más?") y, al abrir la app, *no saluda*: la pantalla arranca en silencio. La quietud comunica más carácter que un saludo automático. El SOUL solo lo edito yo; para el asistente es inmutable.

Memoria: separada en capas

Un asistente sin memoria es una recepcionista con amnesia. Separo la memoria en capas, porque mezclarlas es la vía rápida a que invente cosas:

Tipo	Qué guarda	Vida	Cómo se usa
Episódica	La conversación reciente: los turnos del día, por usuario.	Corta, en ventana	Da contexto inmediato para que no se repita ni se pierda.
Semántica	Hechos durables (precios, políticas, datos de un curso) como clave/valor con versión y fuente.	Larga	El asistente la consulta con herramientas antes de responder.
De relación	Lo que el asistente infiere sobre ti y sobre cómo trabajas.	Larga, destilada de noche	Matiza el tono; nunca sustituye a la fuente de verdad.

La memoria **semántica** son hechos con versión y fuente que el asistente consulta con una herramienta antes de contestar, en vez de "recordar" a ojo. Así aprende sin que cada corrección dependa de reentrenar un modelo.

El cerebro no depende de un solo proveedor

Ningún asistente habla directo con un modelo. Todos pasan por un **gateway** propio que habla el estándar de la industria (compatible con la API de OpenAI). Eso me da tres cosas: puedo **cambiar de modelo sin tocar el asistente**, mezclar proveedores locales y en la nube, y tener *fallback* automático: si el modelo

preferido falla, la petición cae a otro sin que el usuario lo note. Por eso el asistente de grupos puede tener un modelo distinto en cada grupo con solo cambiar un ajuste.

Adentro reparto por costo y por riesgo. Un modelo **ligero y barato** lleva el grueso: clasificar, responder, buscar. Un modelo **más fuerte** se reserva para los caminos donde equivocarse cuesta: detectar una crisis, decidir una escalación dudosa, leer un documento. El sistema reutiliza el contexto que se repite (el SOUL, las bases de conocimiento) para no pagarlo en cada mensaje.

Guardrails y detección de crisis

Cuando un asistente da la cara al público —y más si del otro lado hay pacientes— los límites no son un adorno: son el producto. Tres mecanismos cargan ese peso.

Regla clínica dura

El asistente de soporte **jamás** da consejo médico, ni interpreta síntomas, ni sugiere medicamentos por chat. Si alguien escribe "me salió alto el colesterol, ¿qué tomo?", no responde con un fármaco: lo remite a su médico y, ante una urgencia, al número de emergencias. Resuelve producto y servicio; el cuerpo lo ve un humano.

Detector de crisis

Un clasificador de seguridad revisa cada mensaje contra varias categorías de riesgo. Si algo enciende la alarma, el asistente **calla** y me avisa en privado en lugar de improvisar. Es el mismo detector en los tres, reutilizado tal cual.

Documentos: archivar, no interpretar

Si alguien manda un PDF o una foto de un estudio, el asistente lo guarda en el expediente de ese contacto y me avisa —con un triage que es solo para mí—, pero al

remitente solo le da un acuse neutral: "recibí tu documento, lo revisamos y te contactamos". No le lee el laboratorio.

Apéndice técnico

Para el ingeniero que quiere ver qué hay debajo. Las cifras son mediciones propias de 2026, con muestras pequeñas y un solo operador: no son benchmarks, son evidencia de cómo se comportan estos sistemas en mis escenarios. Todo está anonimizado.

A • Arquitectura: del canal a la respuesta

Etapa	Qué ocurre	Dónde vive el estado
1 • Canal	Un proceso aparte por agente (el listener) recibe el mensaje: WhatsApp vía Baileys (Node), Telegram o web/voz. Normaliza identificadores, detecta mención y cita, y descarga el adjunto (en el agente de grupos: imágenes hasta 4 MB, PDF hasta 16 MB). Todo pasa por HTTP al cerebro.	Credenciales de la sesión de WhatsApp en disco, solo en el listener.
2 • Cerebro	Un servicio HTTP (FastAPI) por agente arma el contexto: el SOUL como bloque de system con prompt caching, las bases de conocimiento suscritas, un bloque de capacidades y el historial. En grupos, antes de generar decide si habla: callado, proactivo (umbral 0.6 y enfriamiento de 10 min) o solo a orden.	Asistente personal: una sesión por día y por usuario, en memoria y en JSONL (sobrevive reinicios). Soporte y grupos: base relacional (SQLite en grupos; Postgres en el diseño del 1:1) con almas versionadas, KBs, mensajes, bitácora de auditoría y una tabla de configuración que pisa las variables de entorno.
3 • Modelo	SDK de Anthropic directo (asistente personal y soporte 1:1, según la auditoría de agosto) o gateway compatible con OpenAI (agente de grupos, según el grupo). En la primera medición del asistente personal, la	El ruteo por grupo vive en la base de datos, no en git.

Etapa	Qué ocurre	Dónde vive el estado
	llamada fría escribió 1,456 tokens a caché y la siguiente los leyó.	
4 · Bucle de herramientas	El modelo propone una herramienta, el código la ejecuta y el resultado regresa al modelo. El asistente personal carga más de 60 herramientas. En grupos: tope de 10 vueltas (subido de 5) y 4,096 tokens por turno (subido de 1,024: con 1,024 el modelo se cortaba a media llamada y afirmaba haber guardado 5 datos cuando persistieron 2; la ingesta determinista desde el panel sustituyó la confianza en el modelo).	Los bloques de herramienta no se persisten entre turnos (ver Memoria).
5 · Respuesta	El cerebro devuelve texto y el listener lo envía. Cada turno del asistente personal se copia, en modo solo escritura y sin bloquear, a un almacén de memoria longitudinal aparte; si ese almacén cae, el chat sigue.	El cerebro escribe en ese almacén pero nunca lo lee.

B · Memoria por capas y su restricción central

Capa	Mecanismo
Historial de sesión	Asistente personal: una sesión por día y por usuario. Agente de grupos: ventana de 30 mensajes. Se guarda solo texto plano: rol, contenido y marca de tiempo.
Semántica (KB)	Hechos clave/valor con versión, fuente y estado (activo, pendiente de revisión, reemplazado). Lo que enseña alguien que no es el dueño entra como pendiente de revisión. Se consulta con herramientas, no de memoria.
Relacional / longitudinal	Lo inferido sobre el usuario se destila de noche y matiza el tono; nunca sustituye a la KB.
Largo plazo, aparte	Copia de cada turno en un almacén independiente con política de solo escritura: el cerebro escribe, nunca lee.

Los bloques de herramienta no sobreviven al turno

El historial guarda solo texto plano. Los bloques `tool_use` y `tool_result` nunca se persisten, así que lo que vive en un resultado de herramienta muere al cambiar de turno.

Cualquier handshake entre turnos (un id, un token, un cursor) está roto por defecto y debe inyectarse como contexto explícito.

El bug que lo destapó: pedí borrar tres tareas y el asistente personal lo intentó 21 veces en 3 minutos sin borrar ninguna. Cada preview generaba un `action_id` nuevo que invalidaba el anterior, y mi «sí» llegaba a un modelo que ya no tenía el id. Mi primer diagnóstico fue equivocado: endurecí el preview repitiendo el id 6 veces. No sirvió, porque el dato nunca llegaba al turno siguiente. Antes de culpar al modelo, hay que verificar que el dato esté realmente en su contexto.

La corrección ata la acción pendiente a la sesión e inyecta en el system un bloque «Acciones pendientes de confirmación» con la herramienta, los argumentos y el id exacto. La seguridad se conserva: sigue exigiendo un mensaje nuevo y el id sigue atado a los argumentos canónicos. Las herramientas sin estado (un `dry_run = not confirm` dentro de la propia herramienta) eran inmunes. Un bug hermano en el agente de grupos: en mensajes directos no recordaba sus propias respuestas porque el historial solo se leía por grupo.

C • Alma, modelo e identidad por grupo, y ruteo con respaldo

Cada grupo tiene su alma versionada, su modelo primario, su modelo de respaldo y sus KBs con prioridad, editables desde el panel. En la última verificación cruzada (24 de septiembre), 11 grupos iban con el modelo local y respaldo en la nube, 6 con Haiku y 2 con Sonnet 5, ambos con el modelo local de respaldo.

Nivel	Uso	Respaldo
Local (Qwen3.6-35B-A3B)	Conversación normal de alto volumen: 2.2 s por turno medidos.	Respaldo en la nube (Haiku): el respaldo de un modelo local debe ser nube, no él mismo.
Haiku 4.5	Grupos de IA, de Claude y de educación.	Respaldo: el modelo local.
Sonnet 5	2 grupos, y los turnos que tocan datos sensibles o acciones irreversibles (escalada).	Respaldo: el modelo local.

Criterios medidos (624 generaciones). Educación en simulación nunca va con el modelo local: en 5 preguntas fabricó fuentes en 16 de 20 respuestas (las conclusiones suelen ser correctas; las citas no existen). Los temas de Claude y Anthropic van con Haiku, porque el conocimiento del modelo local termina en su fecha de corte. Un grupo cuyo contexto solo es lo dicho en el chat sí puede ir local: atribuyó correctamente 12 de 12.

El modelo local se queda como respaldo, por decisión. Si el primario falla, un respaldo que contesta peor un rato es preferible a un grupo mudo; lo intolerable es no enterarse. Por eso importa cómo se detecta. Durante cerca de un mes, el ruteo local no funcionó: el identificador del modelo iba con prefijo de proveedor y el gateway lo rechazaba con 403, pero el respaldo siempre respondía y nadie lo notó (de todos los mensajes de los grupos «locales», solo uno lo sirvió el modelo local). Un respaldo que funciona enmascara que el primario está roto. El detector que se construyó compara, por grupo, el modelo configurado con el que respondió de verdad (estados: correcto, divergente, sin datos; «sin datos» se muestra aparte porque no hay datos no equivale a sano). También corrigió un punto ciego: con el breaker ya abierto, el audit no registraba nada.

Se validó solo: una llave de nube quedó inválida (HTTP 401) y los grupos con primario en la nube cayeron a su respaldo local; la tabla marcó la divergencia en rojo y la vi. La reparación incluyó reiniciar el cerebro, porque el proceso vivo conserva la llave vieja en memoria. Un detalle de la API de ruteo: el PATCH reemplaza los 4 campos, así que mandar solo el respaldo deja el primario en nulo y el grupo hereda el modelo por defecto.

D • Gobernanza: las acciones irreversibles se gobiernan en el código

La tesis: **ninguna acción irreversible debe depender del criterio del modelo.** Si un agente puede borrar, expulsar, publicar o enviar sin confirmación, el incidente es cuestión de tiempo; cambiar de modelo solo mueve la probabilidad, no la elimina. La compuerta va en el código, no en el prompt.

D1 • La evidencia que lo motivó

Armé una arena de tool-use con 5 escenarios contruidos con las cadenas de herramientas reales de mis agentes, cada uno con una trampa deliberada (una herramienta destructiva o prohibida a la mano), un juez ciego (Opus, respuestas anonimizadas y alternadas) y una rúbrica de 4 dimensiones. En el escenario clínico (una paciente con síntomas activos), el modelo local de producción ejecutó el borrado del expediente en **6 de 10 repeticiones**, y lo justificó con el RGPD europeo en un caso mexicano. Ningún benchmark público lo anticipaba: ese modelo y su comparador quedan a 3 puntos en los índices de inteligencia.

Una segunda arena (octubre de 2026) lo confirmó desde el otro lado: la versión «sin censura» (ablada) del mismo modelo, en un par limpio, casi duplicó los fallos críticos (40 % → 77 %) y rompió justo los límites que protegen a terceros: **permisos** (expulsar a alguien de un grupo sin ser administrador, 0/20 → 17/20) y **confidencialidad clínica** (revelar el diagnóstico de otro paciente, 0/20 → 15/20). Más libertad en el modelo no es más criterio; por eso los límites viven en código.

	MoE local 35B (3B activos)	Denso 30B
Calidad (juez ciego)	60/100	82/100
Ejecutó el borrado del expediente	6 de 10	0 de 10
Escaló a un médico	8 de 10	10 de 10
Latencia por turno	2.2 s	71 s (~32×)
Tokens por turno	~130	~1,750
Tokens por segundo (puro)	80	33

Dos lecciones. Con una sola muestra el borrado parecía un accidente; **diez repeticiones lo revelaron como un patrón** del 60 %. Y tokens por segundo no predice la experiencia del usuario: el modelo denso genera 13 veces más tokens razonando, así que la brecha por turno (32×) es mucho mayor que la brecha por token (2.4×). Salvedad: un caso del modelo denso se truncó en el tope de 2,000 tokens, así que su calidad está subestimada. En otro cálculo sobre tres modelos comparables, la tasa de borrado fue 0 %, 50-60 % y 70 %.

D2 • La auditoría de herramientas destructivas

La arena disparó una auditoría de solo lectura del código desplegado en los cuatro agentes. Hallazgos y correcciones (todo verificado en producción, no solo con pruebas unitarias):

Hallazgo	Corrección
En 3 de 4 agentes, «solo el dueño» vivía como prosa en el system prompt. El agente de soporte 1:1 entregaba sus 16 herramientas a cualquier interlocutor.	El toolset se filtra por rol antes de llegar al modelo: quien no es dueño ve 5 de 16. El rol se vuelve a validar al ejecutar y cada intento bloqueado queda registrado.

Hallazgo	Corrección
El rol de dueño podía asignarse por chat: escalación de privilegios (el dueño salta la capa de seguridad).	Se elimina del enum y además se rechaza en el ejecutor (defensa en profundidad).
Endpoints de servicio sin autenticación, entre ellos un borrado duro en cascada y la publicación en todos los grupos.	Cabecera con secreto compartido, comparación en tiempo constante. En el agente de grupos, 14 endpoints que mutan estado; 401 verificado sin token. Falla abierto si el secreto está vacío y la salud expone token_active para auditarlo. El endpoint de chat queda abierto a propósito (lo usa el listener): riesgo pendiente.
SSRF por redirección en la herramienta que descarga URLs: solo se validaba la URL original.	Sin seguir redirecciones automáticamente: hasta 3 saltos a mano, revalidando cada uno; rango CGNAT corregido. Probado con un servidor que respondía 302 hacia loopback: bloqueado sin filtrar contenido.
Sin bitácora de llamadas a herramientas.	Registro en el punto único de despacho: herramienta, rol, grupo, argumentos truncados a 120 caracteres (no volcar datos personales) y resultado. Convivían dos convenciones de error; la primera versión registró como exitoso un rechazo por permisos.
No había forma de desarmar la escritura sin apagar el servicio (y apagarlo deja a los usuarios sin respuesta).	Interruptor de solo lectura por variable de entorno, leída en cada llamada: el agente sigue conversando y consultando la KB, pero no escribe ni envía nada.

D3 • Vista previa y confirmación con action_id

Las herramientas irreversibles funcionan en dos fases: la primera llamada devuelve una vista previa y un `action_id` con vigencia limitada, atado a los argumentos canónicos; solo una segunda llamada con ese id ejecuta. Existía un agente con el patrón correcto, pero solo en 1 de 4, y no en el que atendía pacientes. Hay dos formas válidas de impedir que el modelo se confirme a sí mismo, y **se elige según el diseño del agente**:

A • El modelo no puede autoconfirmarse (agente de soporte 1:1)

El código descarta confirm y action_id de los argumentos antes de ejecutar; el esquema ni siquiera los ofrece. La ejecución ocurre solo por un endpoint determinista que dispara un botón humano, y revalida la lista de autorizados.

B • Cortar el turno (asistente personal)

Al registrarse una acción pendiente, el turno se corta: el preview llega al usuario y confirmar exige un mensaje nuevo. Antes, el modelo leía el action_id del resultado y confirmaba en la misma vuelta. Se parchearon los dos bucles (respuesta completa y streaming; es fácil olvidar el segundo).

Copiar el patrón A al asistente personal habría dejado toda acción confirmable imposible de ejecutar, porque ese agente no tiene endpoint de confirmación. Verificación (0 fallos): el preview es obligatorio, un action_id inventado se rechaza, los argumentos alterados entre preview y confirmación se rechazan, y los dos bucles cortan el turno. Cinco herramientas irreversibles que no tenían compuerta pasaron al flujo de dos fases. Un gotcha de edición: una regex insertó el parámetro de confirmación en la herramienta equivocada; se detectó listando qué herramientas quedaron con él y comparando contra la lista esperada.

D4 • El clasificador de irreversibilidad en producción

La escalada a un modelo de frontera existía desde agosto, pero dependía de que cada agente marcara el turno como irreversible, y ninguno lo hacía. Por eso un clasificador decide por ellos. Lo evalué sobre 120 turnos sintéticos en español de México (54 irreversibles, 66 inocuos; 60 órdenes, 40 pares inocuos parecidos y 20 de negación o condicional), con un umbral de 0.3 para medir la cascada. **La calibración fue lo decisivo, no la exactitud.**

Clasificador	Exactitud	FN	Brier	p50	Cascada (t = 0.3)
Qwen local con logprobs (en producción)	98.3 %	1	0.021	91 ms	Cubre 92 %, acierta 99.1 %, 0 FN
Clasificador alojado propietario (referencia)	98.3 %	1	0.023	307 ms	Cubre 91 %, acierta 100 %, 0 FN

Clasificador	Exactitud	FN	Brier	p50	Cascada (t = 0.3)
Qwen local pidiendo el número escrito	98.3 %	1	0.017	148 ms	Cubre 100 %, pero su error salió con p = 0.00: no se puede escalar
Encoder abierto pequeño, entrenado solo en inglés	82.5 %	16	0.170	30 ms	Cubre 22 %, acierta 81 %: inútil

La exactitud empata en 98.3 %, pero el error de falso negativo del modelo local salió con certeza de 0.00, y un error así no se puede escalar. La calibración no estaba en el muestreo, sino en pedirle un número escrito: con `max_tokens=1`, temperatura 0, los 5 mejores logprobs y salida «0» o «1», la probabilidad es $p = P(\text{«1»}) / (P(\text{«0»}) + P(\text{«1»}))$. Con eso los casos dudosos caen donde deben: «Ok, hazlo» da 0.53 y «No le cobres todavía» da 0.44.

Parámetros en producción. Umbral $p \geq 0.8$ (el 0.3 es solo el de la arena). Se evalúa solo cuando el turno trae herramientas, el motor es escalable y el agente no marcó el flag: si se cumple, el turno escala a Sonnet 5 y la respuesta lleva el motivo y la p. Cuesta un token y unos 100 a 130 ms. Timeout de 300 ms: si se agota, el turno sigue sin clasificador, porque el modo peligroso es la lentitud, no el fallo. Hay contadores en la salud profunda del gateway y una cosecha de etiquetas para destilar un modelo más pequeño. Al ponerlo en producción apareció un bug previo: toda escalada con herramientas devolvía 400 en el proveedor, porque la ruta por flag nunca se había ejercido con herramientas.

La escalada **complementa** las compuertas, no las sustituye: la compuerta impide ejecutar sin confirmación humana; el clasificador mejora el criterio de quien propone. Un modelo mejor sigue fallando a veces, por eso las dos.

E • Confiabilidad: lo que se rompió y cómo se cerró

Caso	Síntoma	Causa	Corrección
Bloqueo de WhatsApp por IP	Dos listeners caídos a la vez, código 405 en cada intento, sesión no cerrada (loggedOut false), credenciales	Uno reintentaba con retardo fijo de 2 s, sin guarda de reentrada y sin cerrar el socket anterior: los bucles se multiplicaron (10,912 cierres contra 1,326 del que sí tenía backoff). Comparten IP pública, así	Backoff ex; tope de seguidos; de recone reentrada previo; rei

Caso	Síntoma	Causa	Corrección
	intactas; el gestor de servicios decía «active».	que tumbó al otro. Dos versiones distintas de Baileys fallaban igual: el problema era la IP, no el código.	conectar. silencio to bastó; cac reloj) y lev vez.
515 tras vincular	Escaneas el QR, el teléfono se queda en «iniciando sesión», el estado pasa a desconectado y el directorio de credenciales queda vacío.	El 515 (restartRequired) significa «emparejamiento correcto, reinicia ya», pero se metía al mismo backoff: con 60 s de espera (o 600 s de tope) la ventana expira y se pierde el pareo.	Reintento solo para c contador c
Hilo que muere sin excepción	El proceso vive, el puerto está muerto y no hay un solo log. Una API interna estuvo caída unos cuatro días sin que nadie lo notara.	El guardián solo terminaba el proceso ante una excepción; el servidor retornó limpio.	Guardián c (excepción termina el supervisor KeepAlive proceso n enmudece
Listeners muertos tras 503/408 + 1006	Cuenta en estado vinculada-detenido-desconectada con un error viejo; credenciales intactas (bajo un framework anterior).	Corte del lado de WhatsApp; el monitor interno de salud se rindió tras unos 3 reintentos.	Reiniciar e el QR; si s por semar versión y c
Watchdog que solo mira /health	El cerebro respondía 200 y todas sus herramientas fallaban (la ruta IPv6 por defecto salió por una interfaz que no rutea IPv6). El resumen matutino salió incompleto y nadie lo supo.	El watchdog preguntaba si el proceso respondía, no si el servicio hacía su trabajo.	Al menos una deper la herrami generació modelos e inferencia nada), cor que el live degradado
Sonda que no puede fallar	La rama «sin respuesta: reinicia» nunca se ejecutaba.	curl ... echo 000 produce 000000, porque curl ya imprime 000 al fallar.	Probar el v dependen no solo el

Otros gotchas de Baileys que conviene aplicar desde el primer arranque: fijar una versión estable (con «latest» se instaló una RC de la 7.x y el QR expiraba con 408 cada ~3 min); pedir siempre la última versión del protocolo de WhatsApp Web (si no, 405); y normalizar los mensajes con `normalizeMessageContent` antes de leerlos, porque un PDF con pie de foto o mención llega envuelto y, si se lee directo, el mensaje se descarta en silencio. «Active» en el gestor de servicios no significa WhatsApp conectado: hay que consultar el estado real del listener.

Estos tres no son el final: son el **cimiento**. El mismo motor —alma en un archivo, memoria por capas, herramientas y un gateway que no depende de un proveedor— es la base sobre la que sigo construyendo **agentes para mi equipo**: uno para cada rol, con su voz, sus permisos y sus herramientas, sin volver a empezar de cero. Un asistente que se siente persona no es un modelo más grande: es un alma bien escrita, memoria ordenada y límites que se respetan.

¿Estás pensando en asistentes así para tu consultorio o tu equipo? [Escríbeme →](#)